Changelog — v3.0 (2026-04-04)

> Performance-focused architecture optimization. Eliminates the MVCC_READ_CONFLICT write bottleneck, adds CouchDB indexes, secures bootstrap, and reduces block-cut latency — validated by Hyperledger Caliper benchmarks showing a **63× write throughput improvement**.

Summary

Metric	v2.0 (Previous)	v3.0 (Current)	Improvement
Write success rate (concurrent)	10%	**100%**	MVCC_READ_CONFLICT eliminated
Max verified write throughput	0.8 TPS	**50.5 TPS**	**63× faster**
Serial write latency	2.09s	**0.10s**	**21× lower**
Write latency floor (block cut)	2.0s	**0.5s**	**75% reduction**
Point read throughput	>2,000 TPS	>2,000 TPS	No regression
CouchDB indexes	0	2 composite indexes	New
ID generation	Sequential counter (shared key)	SHA-256 deterministic hash	New
InitLedger access control	None	Caller MSP validation	New

Architecture Decision Records (ADRs)

Five ADRs were proposed during the agency-blueprint evaluation of v2.0. Four were implemented in v3.0:

ADR	Title	Status	Impact
ADR-001	Replace sequential counters with hash-based IDs	☑ Implemented	**CRITICAL** — eliminated MVCC_READ_CONFLICT
ADR-002	Add CouchDB composite indexes	☑ Implemented	HIGH — improves filtered query performance
ADR-003	Add server-side pagination	☐ Deferred to v3.1	MEDIUM — needed for large datasets
ADR-004	Secure InitLedger with MSP validation	☑ Implemented	HIGH — closes bootstrap exploit

ADR	Title	Status	Impact
ADR-005	Reduce BatchTimeout from 2s to 500ms	☑ Implemented	HIGH — 75% write latency reduction

Chaincode Changes (Go)

ADR-001: Hash-Based Deterministic ID Generation

Root cause: The `GetNextID()` function in `assets/counter.go` maintained a shared on-chain counter key (e.g., `counter_eGO`) that every write transaction read and incremented. When multiple transactions landed in the same block, all but the first were invalidated at commit time due to MVCC_READ_CONFLICT (Fabric status code 11). This limited write throughput to **~0.8 TPS** regardless of network capacity.

Solution: New `GenerateID()` function derives IDs deterministically from the transaction ID:

```
// assets/counter.go — NEW
func GenerateID(ctx contractapi.TransactionContextInterface, prefix string, suffix
int) (string, error) {
    txID := ctx.GetStub().GetTxID()
    raw := txID + "_" + strconv.Itoa(suffix)
    hash := sha256.Sum256([]byte(raw))
    return prefix + hex.EncodeToString(hash[:8]), nil  // e.g.,
"eGO_a1b2c3d4e5f6a7b8"
}
```

Each transaction's ID is unique (guaranteed by Fabric), so every `GenerateID()` call produces a globally unique key with zero shared-state reads. The `suffix` parameter disambiguates multiple IDs generated within one transaction (e.g., cancellation creates both a certificate and a remainder GO).

Prefix constants and range-end constants:

```
const (
    PrefixDevice        = "device_"
    PrefixEGO           = "eGO_"
    PrefixHGO           = "hGO_"
    PrefixECancellation = "eCancel_"
    PrefixHCancellation = "hCancel_"
    PrefixEConsumption  = "eCon_"
    PrefixHConsumption  = "hCon_"

    RangeEndEGO    = "eGO_~"        // CouchDB sorts '~' after all hex chars
    RangeEndHGO    = "hGO_~"
    RangeEndDevice = "device_~"
)
```

Files modified:

File	Change
`chaincode/assets/counter.go`	Added `GenerateID()`, prefix/range-end constants. `GetNextID()` retained but marked DEPRECATED
`chaincode/contracts/device_mgmt.go`	`RegisterDevice` uses `GenerateID(ctx, PrefixDevice, 0)`
`chaincode/contracts/issuance.go`	`CreateElectricityGO` → `GenerateID(ctx, PrefixEGO, 0)`, `CreateHydrogenGO` → `GenerateID(ctx, PrefixHGO, 0)`
`chaincode/util/split.go`	`SplitElectricityGO` remainder → `GenerateID(ctx, PrefixEGO, 0)`, `SplitHydrogenGO` remainder → `GenerateID(ctx, PrefixHGO, 0)`
`chaincode/contracts/cancellation.go`	Per-transaction `suffixCounter` incremented per `GenerateID` call: cancellation key (N), consumption declaration (N+1), remainder GO (N+2)
`chaincode/contracts/conversion.go`	`IssuehGO` loop uses incrementing suffix counter for consumption keys, remainder eGOs, and final hGO
`chaincode/contracts/query.go`	`GetCurrentEGOsList` range → `"eGO"/"eGO~"`, `GetCurrentHGOsList` range → `"hGO"/"hGO~"` — captures both legacy (`eGO1`) and new (`eGO_hash`) format keys

ID format change:

Entity	v2.0 Format	v3.0 Format	Example
Device	`device1`, `device2`, ...	`device_<16-hex>`	`device_a1b2c3d4e5f6a7b8`
Electricity GO	`eGO1`, `eGO2`, ...	`eGO_<16-hex>`	`eGO_f9e8d7c6b5a49382`
Hydrogen GO	`hGO1`, `hGO2`, ...	`hGO_<16-hex>`	`hGO_1a2b3c4d5e6f7a8b`
E-Cancellation	`eCancellation1`	`eCancel_<16-hex>`	`eCancel_112233aabbccddee`
H-Cancellation	`hCancellation1`	`hCancel_<16-hex>`	`hCancel_ffeeddccbbaa9988`
E-Consumption	`eConsumption1`	`eCon_<16-hex>`	`eCon_0011223344556677`
H-Consumption	`hConsumption1`	`hCon_<16-hex>`	`hCon_8899aabbccddeeff`

Backward compatibility: Range queries in `query.go` use start key `"eGO"` (not `"eGO_"`), which captures both old sequential keys (`eGO1`) and new hash-based keys (`eGO_abc...`). Mixed-format ledgers remain queryable during migration.

ADR-002: CouchDB Composite Indexes

Two composite indexes added to accelerate filtered queries against CouchDB:

New files:

```
chaincode/META-INF/statedb/couchdb/indexes/
├── indexOwner.json     — Composite index on [OwnerID, GOType]
└── indexGOType.json    — Composite index on [GOType, CreationDateTime]
```

indexOwner.json:

```
{
  "index": {"fields": ["OwnerID", "GOType"]},
  "ddoc": "indexOwnerDoc",
  "name": "indexOwner",
  "type": "json"
}
```

indexGOType.json:

```
{
  "index": {"fields": ["GOType", "CreationDateTime"]},
  "ddoc": "indexGOTypeDoc",
  "name": "indexGOType",
  "type": "json"
}
```

These indexes are automatically deployed to all CouchDB instances when the chaincode is installed. They improve performance for:

- Querying GOs owned by a specific organization
- Filtering GOs by type and sorting by creation date

ADR-004: Secure InitLedger

Vulnerability in v2.0: `InitLedger(issuerMSP)` had no caller validation. Any organization could call it to register any MSP as the network issuer, enabling privilege escalation.

Fix: Added caller MSP validation:

```go
func (c *DeviceContract) InitLedger(ctx contractapi.TransactionContextInterface,
issuerMSP string) error {
    callerMSP, err := ctx.GetClientIdentity().GetMSPID()
    if err != nil {
        return fmt.Errorf("failed to get caller MSPID: %v", err)
    }
    if callerMSP != issuerMSP {
        return fmt.Errorf("access denied: caller MSP %s cannot register a
```

```
different org %s as issuer",
            callerMSP, issuerMSP)
    }
    return access.RegisterOrgRole(ctx, issuerMSP, access.RoleIssuer)
}
```

Now each organization can only register itself — an org calling `InitLedger("otherOrgMSP")` is rejected.

🌐 Network Configuration Changes

ADR-005: BatchTimeout Reduction (2s → 500ms)

File: `network/configtx.yaml`

```
Orderer:
    BatchTimeout: 500ms    # Was: 2s
    BatchSize:
        MaxMessageCount: 10
        AbsoluteMaxBytes: 99 MB
        PreferredMaxBytes: 512 KB
```

Impact: The write latency floor dropped from 2.0s to 0.5s. In the v3.0 benchmark, measured average write latency is 0.10s — meaning most transactions commit within the first 500ms block window rather than waiting up to 2 seconds.

Orderer TLS Certificate Path Fixes

Files: `network/configtx.yaml`, `network/docker/docker-compose-orderer.yaml`

Fixed certificate paths to use the correct cryptogen-generated hostnames:

Component	v2.0 (incorrect)	v3.0 (correct)
Orderer MSP/TLS	`ordererN.go-platform.com/`	`ordererN.orderer.go-platform.com/`

This resolved orderer boot failures where TLS certificates could not be found at the expected paths.

Deploy Chaincode Script

File: `network/scripts/deploy-chaincode.sh`

- Bumped `CHAINCODE_VERSION` from `"2.0"` to `"3.0"`
- Fixed contract namespace references: `DeviceContract:InitLedger` → `device:InitLedger`, `DeviceContract:RegisterOrgRole` → `device:RegisterOrgRole`
- Fixed test query: `DeviceContract:GetAllDevices` → `device:ListDevices`

📊 Testing & Benchmarking (New)

New Caliper Benchmark Infrastructure

File	Purpose
`testing/caliper-workspace/network-config.yaml`	Caliper network descriptor — 3 orgs, fabric-gateway binding
`testing/caliper-workspace/connection-profiles/ccp-*.yaml`	Connection profiles for issuer1, eproducer1, buyer1
`testing/caliper-workspace/bench-config.yaml`	16-round read scalability benchmark (GetDevice, ListDevices, GetCurrentEGOsList at 100–2000 TPS)
`testing/caliper-workspace/bench-write-v2.yaml`	5-round write scalability benchmark (RegisterDevice serial through 50 TPS)
`testing/caliper-workspace/bench-seed.yaml`	Device seeding config (30 devices via fixed-load)
`testing/caliper-workspace/workload/registerDevice.js`	Write workload — registers devices with transient data
`testing/caliper-workspace/workload/getDevice.js`	Point-read workload — discovers device IDs from JSON file (supports hash-based IDs)
`testing/caliper-workspace/workload/listDevices.js`	Range-query workload — calls `device:ListDevices`
`testing/caliper-workspace/workload/getCurrentEGOsList.js`	Range-query workload — calls `query:GetCurrentEGOsList`
`testing/caliper-workspace/workload/readPublicEGO.js`	Public data read workload
`testing/PERFORMANCE_REPORT.md`	Comprehensive before/after benchmark report
`testing/ARCHITECTURE_EVALUATION.md`	Agency-blueprint evaluation with 5 ADR proposals

Benchmark Results Summary

Write Performance (the critical improvement):

Round	Txns	Success	Fail	Success Rate	Avg Latency	Throughput
RegisterDevice-serial	30	30	0	**100%**	0.10s	5.7 TPS
RegisterDevice-5tps	50	50	0	**100%**	0.09s	6.2 TPS
RegisterDevice-10tps	100	100	0	**100%**	0.09s	11.0 TPS
RegisterDevice-25tps	250	250	0	**100%**	0.10s	25.8 TPS
RegisterDevice-50tps	500	500	0	**100%**	0.10s	50.5 TPS

Read Performance (unchanged from v2.0):

Round	Txns	Success	Fail	Avg Latency	Throughput
GetDevice-2000tps	10,000	10,000	0	<0.01s	1,996.4 TPS
GetCurrentEGOsList-1000tps	5,000	5,000	0	0.01s	998.6 TPS

File Inventory

New Files (15)

Category	Count	Files
CouchDB indexes	2	`chaincode/META-INF/statedb/couchdb/indexes/indexOwner.json`, `indexGOType.json`
Testing infrastructure	8	`bench-config.yaml`, `bench-write-v2.yaml`, `bench-seed.yaml`, `network-config.yaml`, 4× workload JS files
Testing docs	2	`PERFORMANCE_REPORT.md`, `ARCHITECTURE_EVALUATION.md`
Connection profiles	3	`ccp-issuer1.yaml`, `ccp-eproducer1.yaml`, `ccp-buyer1.yaml`

Modified Files (10)

File	Change Summary
`chaincode/assets/counter.go`	+`GenerateID()`, +prefix/range constants, `GetNextID` deprecated
`chaincode/contracts/device_mgmt.go`	Hash-based device IDs, secure `InitLedger`
`chaincode/contracts/issuance.go`	Hash-based eGO/hGO IDs
`chaincode/contracts/cancellation.go`	Suffix counter for multi-ID transactions
`chaincode/contracts/conversion.go`	Suffix counter for loop-based ID generation
`chaincode/contracts/query.go`	Updated range-query bounds for hash-based keys
`chaincode/util/split.go`	Hash-based remainder IDs
`network/configtx.yaml`	BatchTimeout 2s→500ms, fixed orderer TLS paths
`network/docker/docker-compose-orderer.yaml`	Fixed orderer volume mount paths
`network/scripts/deploy-chaincode.sh`	Version bump to 3.0, fixed contract name references

⚠ Breaking Changes

1. **Device IDs are no longer sequential.** Any client code that assumes `device1`, `device2`, etc. must switch to querying `device:ListDevices` and using the returned IDs.

2. **All GO asset IDs are now hash-based.** Clients must use the `AssetID` returned from `CreateElectricityGO` / `CreateHydrogenGO` responses rather than constructing IDs from sequential counters.

3. **`InitLedger` now validates caller MSP.** Each organization must call `InitLedger` with its own MSP ID; cross-org registration is rejected.

4. **BatchTimeout reduced to 500ms.** Block production is more frequent, which slightly increases orderer I/O. Monitor orderer memory and disk usage in production.

➡ SOON Roadmap (v3.1 and beyond)

- ☐ **ADR-003: Server-side pagination** — `ListDevices`, `GetCurrentEGOsList`, `GetCurrentHGOsList` with bookmark-based pagination (max 50 records per page)
- ☐ Write throughput ceiling test — push RegisterDevice to 100, 200, 500 TPS to find actual ceiling
- ☐ Multi-VM distributed deployment benchmark
- ☐ Chaincode unit tests for `GenerateID()` collision analysis
- ☐ Migration tool for existing v2.0 sequential IDs to v3.0 hash format
- ☐ Caliper benchmark for full GO lifecycle (issuance + transfer + cancellation)
- ☐ Production-grade CouchDB credentials management (remove hardcoded `admin/adminpw`)

Committed on 2026-04-04 — v3.0 performance optimization release.